Syntax Analysis, Structural Testing, & Test Cases

Software Development

Guided Exercise 2b

Universidad Carlos III de Madrid

Team: T03

Members: Pablo Velarde and Ellis Kim

Date: April 24, 2026

Contents

1 Introduction **2**

2 Syntax Analysis **2**

2.1 Testing strategy 2
2.2 Grammar 2
2.3 Derivation tree 3
2.4 Detailed explanation of the 79 syntax analysis test cases 3
2.5 Justification of the syntax analysis test suite 9

3 Structural Testing **9**

3.1 Testing strategy 9
3.2 Flow control graph 9
3.3 Basic paths 11
3.4 Detailed explanation of the 12 structural testing test cases 12
3.5 Additional cases for loops 14
3.6 Justification of the structural test suite 14

4 Conclusion **14**

1 Introduction

This document defines the test cases for Method 2 of Guided Exercise 2b, `register_document`. The purpose of this document is to explain the syntax analysis and structural testing applied to the method, justify the selected test cases, and document the grammar, derivation tree, flow control graph, mccabe complexity, and basic paths used during the test design process. According to the assignment handout, the method `register_document(input_file: str)` receives the path to a JSON input file with the following expected structure:

```
{
  "PROJECT_ID": "32 hexadecimal characters",
  "FILENAME": "<NAME><EXTENSION>"
}
```

where:

- `PROJECT_ID` must be a valid 32-character hexadecimal string.
- `FILENAME` must contain:
  - a name of exactly 8 alphanumeric characters
  - one valid extension: `.pdf`, `.docx`, or `.xlsx`

If the input data is valid, the method must instantiate a `ProjectDocument`, generate a SHA-256 signature, store the information in `all_documents.json`, and return the signature in hexadecimal format. Otherwise, it must raise an `EnterpriseManagementException` according to the type of detected error.

2 Syntax Analysis

2.1 Testing strategy

The syntax analysis technique was applied because the input of Method 2 can be modeled using a grammar. Following the procedure explained in class, we first defined the grammar for valid input files, then constructed the corresponding derivation tree, and finally defined test cases to cover the valid and invalid cases derived from that tree. The first objective was to define valid test cases that cover all non-terminal nodes and, afterwards, all remaining terminal nodes. Then, invalid test cases were defined by applying deletion, duplication, and modification operations to the nodes of the derivation tree. As explained in class, some invalid cases were not repeated when they were already covered by equivalent mutations of upper-level nodes or by non-terminal mutations that produced the same resulting input.

2.2 Grammar

The grammar defined for the valid JSON input is the following:

```
<File> ::= <Start_Object> <Data> <End_Object>
<Start_Object> ::= {
<End_Object> ::= }
```

```
<Data> ::= <Field1> <Separator> <Field2>

<Field1> ::= <Label_Data_1> <Equal> <Value_Data_1>
<Field2> ::= <Label_Data_2> <Equal> <Value_Data_2>

<Separator> ::= ,
<Equal> ::= :
<Quotation_Marks> ::= "

<Label_Data_1> ::= <Quotation_Marks> <Value_Label_1> <Quotation_Marks>
<Value_Label_1> ::= PROJECT_ID
<Value_Data_1> ::= <Quotation_Marks> <Value_1> <Quotation_Marks>
<Value_1> ::= a|b|c|d|e|f|0|1|...|9 (x32)

<Label_Data_2> ::= <Quotation_Marks> <Value_Label_2> <Quotation_Marks>
<Value_Label_2> ::= FILENAME
<Value_Data_2> ::= <Quotation_Marks> <Value_2> <Quotation_Marks>
<Value_2> ::= <Name> <Extension>
<Name> ::= alphanumeric_character repeated 8 times
<Extension> ::= a|...|z|A|...|Z|0|...|9 (x8)
```

2.3 Derivation tree

Figure 1 shows the derivation tree constructed from the grammar above. As required in class, the nodes were numbered from top to bottom and from left to right.

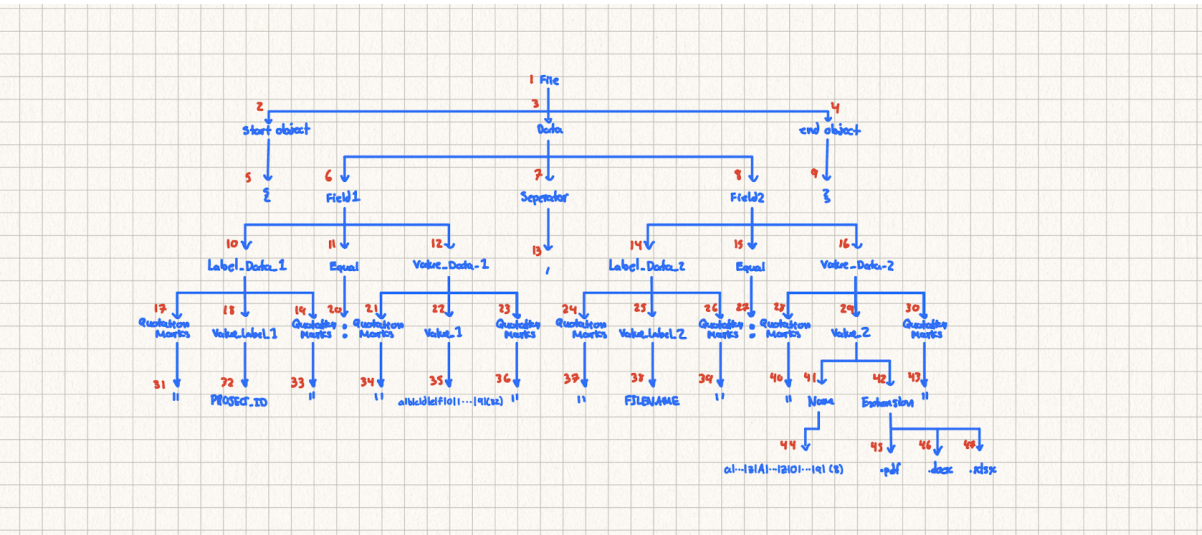

Figure 1: Derivation tree for Method 2 input grammar

2.4 Detailed explanation of the 79 syntax analysis test cases

Table 1 contains the full list of the 79 test cases defined for Method 2. For each test case, we indicate its objective, the relevant mutated or covered node(s), and the expected result. For the

valid test cases, the SHA-256 signature was omitted, as its length is too large to display in the table; however, the correct signatures are located in the Excel file.

Table 1: Explanation of the 79 test cases for Method 2

Test	Objective / Explanation	Node(s)	Expected result
TC1	Valid JSON with a valid `PROJECT_ID` and a valid `FILENAME` ending in `.pdf`. Chosen to cover all non-terminal nodes and most terminal nodes.	1–45	Valid SHA-256 signature returned and data stored in `all_documents.json`
TC2	Valid JSON with a valid `PROJECT_ID` and a valid `FILENAME` ending in `.docx`. Added to cover the missing valid terminal for `.docx`.	46	Valid SHA-256 signature returned and data stored in `all_documents.json`
TC3	Valid JSON with a valid `PROJECT_ID` and a valid `FILENAME` ending in `.xlsx`. Added to cover the missing valid terminal for `.xlsx`.	47	Valid SHA-256 signature returned and data stored in `all_documents.json`
TC4	Duplication of node 1 (`File`). The whole valid JSON object is duplicated, producing two consecutive JSON objects.	1	`EnterpriseManagementException`: The file is not JSON formatted
TC5	Deletion of node 1 (`File`). The entire JSON object is removed, leaving an empty file.	1	`EnterpriseManagementException`: The file is not JSON formatted
TC6	Duplication of node 2 (`Start_Object`). The opening brace is duplicated.	2, 5	`EnterpriseManagementException`: The file is not JSON formatted
TC7	Deletion of node 2 (`Start_Object`). The opening brace is removed.	2, 5	`EnterpriseManagementException`: The file is not JSON formatted
TC8	Duplication of node 3 (`Data`). The whole data block is duplicated.	3	`EnterpriseManagementException`: The file is not JSON formatted
TC9	Deletion of node 3 (`Data`). The data block is removed, leaving an empty JSON object.	3	`EnterpriseManagementException`: JSON does not have the expected structure
TC10	Duplication of node 4 (`End_Object`). The closing brace is duplicated.	4, 9	`EnterpriseManagementException`: The file is not JSON formatted
TC11	Deletion of node 4 (`End_Object`). The closing brace is removed.	4, 9	`EnterpriseManagementException`: The file is not JSON formatted
TC12	Duplication of node 6 (`Field1`). The `PROJECT_ID` field is duplicated.	6	`EnterpriseManagementException`: The file is not JSON formatted

Test	Objective / Explanation	Node(s)	Expected result
TC13	Deletion of node 6 (`Field1`). The `PROJECT_ID` field is removed.	6	`EnterpriseManagementException`: The file is not JSON formatted
TC14	Duplication of node 7 (`Separator`). The comma between both fields is duplicated.	7, 13	`EnterpriseManagementException`: The file is not JSON formatted
TC15	Deletion of node 7 (`Separator`). The comma between both fields is removed.	7, 13	`EnterpriseManagementException`: The file is not JSON formatted
TC16	Duplication of node 8 (`Field2`). The `FILENAME` field is duplicated.	8	`EnterpriseManagementException`: The file is not JSON formatted
TC17	Deletion of node 8 (`Field2`). The `FILENAME` field is removed.	8	`EnterpriseManagementException`: The file is not JSON formatted
TC18	Duplication of node 10 (`Label_Data_1`). The full `PROJECT_ID` label block is duplicated.	10	`EnterpriseManagementException`: The file is not JSON formatted
TC19	Deletion of node 10 (`Label_Data_1`). The full `PROJECT_ID` label block is removed.	10	`EnterpriseManagementException`: The file is not JSON formatted
TC20	Duplication of node 11 (`Equal`). The colon after `PROJECT_ID` is duplicated.	11, 20	`EnterpriseManagementException`: The file is not JSON formatted
TC21	Deletion of node 11 (`Equal`). The colon after `PROJECT_ID` is removed.	11, 20	`EnterpriseManagementException`: The file is not JSON formatted
TC22	Duplication of node 12 (`Value_Data_1`). The entire `PROJECT_ID` value block is duplicated.	12	`EnterpriseManagementException`: The file is not JSON formatted
TC23	Deletion of node 12 (`Value_Data_1`). The entire `PROJECT_ID` value block is removed.	12	`EnterpriseManagementException`: The file is not JSON formatted
TC24	Duplication of node 14 (`Label_Data_2`). The full `FILENAME` label block is duplicated.	14	`EnterpriseManagementException`: The file is not JSON formatted
TC25	Deletion of node 14 (`Label_Data_2`). The full `FILENAME` label block is removed.	14	`EnterpriseManagementException`: The file is not JSON formatted
TC26	Duplication of node 15 (`Equal`). The colon after `FILENAME` is duplicated.	15, 27	`EnterpriseManagementException`: The file is not JSON formatted

Test	Objective / Explanation	Node(s)	Expected result
TC27	Deletion of node 15 (`Equal`). The colon after `FILENAME` is removed.	15, 27	`EnterpriseManagementException`: The file is not JSON formatted
TC28	Duplication of node 16 (`Value_Data_2`). The entire `FILENAME` value block is duplicated.	16	`EnterpriseManagementException`: The file is not JSON formatted
TC29	Deletion of node 16 (`Value_Data_2`). The entire `FILENAME` value block is removed.	16	`EnterpriseManagementException`: The file is not JSON formatted
TC30	Duplication of node 17 (`Quotation Marks`). The opening quotation mark of the `PROJECT_ID` label is duplicated.	17, 31	`EnterpriseManagementException`: The file is not JSON formatted
TC31	Deletion of node 17 (`Quotation Marks`). The opening quotation mark of the `PROJECT_ID` label is removed.	17, 31	`EnterpriseManagementException`: The file is not JSON formatted
TC32	Duplication of node 18 (`Value_Label_1`). The text `PROJECT_ID` is duplicated.	18, 32	`EnterpriseManagementException`: JSON does not have the expected structure
TC33	Deletion of node 18 (`Value_Label_1`). The text `PROJECT_ID` is removed.	18, 32	`EnterpriseManagementException`: JSON does not have the expected structure
TC34	Duplication of node 19 (`Quotation Marks`). The closing quotation mark of the `PROJECT_ID` label is duplicated.	19, 33	`EnterpriseManagementException`: The file is not JSON formatted
TC35	Deletion of node 19 (`Quotation Marks`). The closing quotation mark of the `PROJECT_ID` label is removed.	19, 33	`EnterpriseManagementException`: The file is not JSON formatted
TC36	Duplication of node 21 (`Quotation Marks`). The opening quotation mark of the `PROJECT_ID` value is duplicated.	21, 34	`EnterpriseManagementException`: The file is not JSON formatted
TC37	Deletion of node 21 (`Quotation Marks`). The opening quotation mark of the `PROJECT_ID` value is removed.	21, 34	`EnterpriseManagementException`: The file is not JSON formatted
TC38	Duplication of node 22 (`Value_1`). The `PROJECT_ID` value text is duplicated.	22, 35	`EnterpriseManagementException`: JSON data has no valid values
TC39	Deletion of node 22 (`Value_1`). The `PROJECT_ID` value text is removed.	22, 35	`EnterpriseManagementException`: JSON data has no valid values
TC40	Duplication of node 23 (`Quotation Marks`). The closing quotation mark of the `PROJECT_ID` value is duplicated.	23, 36	`EnterpriseManagementException`: The file is not JSON formatted

Test	Objective / Explanation	Node(s)	Expected result
TC41	Deletion of node 23 (`Quotation Marks`). The closing quotation mark of the `PROJECT_ID` value is removed.	23, 36	`EnterpriseManagementException`: The file is not JSON formatted
TC42	Duplication of node 24 (`Quotation Marks`). The opening quotation mark of the `FILENAME` label is duplicated.	24, 37	`EnterpriseManagementException`: The file is not JSON formatted
TC43	Deletion of node 24 (`Quotation Marks`). The opening quotation mark of the `FILENAME` label is removed.	24, 37	`EnterpriseManagementException`: The file is not JSON formatted
TC44	Duplication of node 25 (`Value_Label_2`). The text `FILENAME` is duplicated.	25, 38	`EnterpriseManagementException`: JSON does not have the expected structure
TC45	Deletion of node 25 (`Value_Label_2`). The text `FILENAME` is removed.	25, 38	`EnterpriseManagementException`: JSON does not have the expected structure
TC46	Duplication of node 26 (`Quotation Marks`). The closing quotation mark of the `FILENAME` label is duplicated.	26, 39	`EnterpriseManagementException`: The file is not JSON formatted
TC47	Deletion of node 26 (`Quotation Marks`). The closing quotation mark of the `FILENAME` label is removed.	26, 39	`EnterpriseManagementException`: The file is not JSON formatted
TC48	Duplication of node 28 (`Quotation Marks`). The opening quotation mark of the `FILENAME` value is duplicated.	28, 40	`EnterpriseManagementException`: The file is not JSON formatted
TC49	Deletion of node 28 (`Quotation Marks`). The opening quotation mark of the `FILENAME` value is removed.	28, 40	`EnterpriseManagementException`: The file is not JSON formatted
TC50	Duplication of node 29 (`Value_2`). The full `FILENAME` value content is duplicated.	29	`EnterpriseManagementException`: JSON data has no valid values
TC51	Deletion of node 29 (`Value_2`). The full `FILENAME` value content is removed.	29	`EnterpriseManagementException`: JSON data has no valid values
TC52	Duplication of node 30 (`Quotation Marks`). The closing quotation mark of the `FILENAME` value is duplicated.	30, 43	`EnterpriseManagementException`: The file is not JSON formatted
TC53	Deletion of node 30 (`Quotation Marks`). The closing quotation mark of the `FILENAME` value is removed.	30, 43	`EnterpriseManagementException`: The file is not JSON formatted
TC54	Duplication of node 41 (`Name`). The file name part is duplicated.	41, 44	`EnterpriseManagementException`: JSON data has no valid values
TC55	Deletion of node 41 (`Name`). The file name part is removed.	41, 44	`EnterpriseManagementException`: JSON data has no valid values

Test	Objective / Explanation	Node(s)	Expected result
TC56	Duplication of node 42 (`Extension`). The extension is duplicated.	42, 45	`EnterpriseManagementException`: JSON data has no valid values
TC57	Deletion of node 42 (`Extension`). The extension is removed.	42, 45, 46, 47	`EnterpriseManagementException`: JSON data has no valid values
TC58	Modification of node 5. The opening brace is replaced with another character.	5	`EnterpriseManagementException`: The file is not JSON formatted
TC59	Modification of node 9. The closing brace is replaced with another character.	9	`EnterpriseManagementException`: The file is not JSON formatted
TC60	Modification of node 13. The separator comma is replaced with another character.	13	`EnterpriseManagementException`: The file is not JSON formatted
TC61	Modification of node 20. The colon after `PROJECT_ID` is replaced with another character.	20	`EnterpriseManagementException`: The file is not JSON formatted
TC62	Modification of node 27. The colon after `FILENAME` is replaced with another character.	27	`EnterpriseManagementException`: The file is not JSON formatted
TC63	Modification of node 31. The opening quotation mark of the `PROJECT_ID` label is replaced with another character.	31	`EnterpriseManagementException`: The file is not JSON formatted
TC64	Modification of node 32. The text `PROJECT_ID` is changed to a different label.	32	`EnterpriseManagementException`: JSON does not have the expected structure
TC65	Modification of node 33. The closing quotation mark of the `PROJECT_ID` label is replaced with another character.	33	`EnterpriseManagementException`: The file is not JSON formatted
TC66	Modification of node 34. The opening quotation mark of the `PROJECT_ID` value is replaced with another character.	34	`EnterpriseManagementException`: The file is not JSON formatted
TC67	Modification of node 35. The `PROJECT_ID` value is changed so it is no longer valid.	35	`EnterpriseManagementException`: JSON data has no valid values
TC68	Modification of node 36. The closing quotation mark of the `PROJECT_ID` value is replaced with another character.	36	`EnterpriseManagementException`: The file is not JSON formatted
TC69	Modification of node 37. The opening quotation mark of the `FILENAME` label is replaced with another character.	37	`EnterpriseManagementException`: The file is not JSON formatted
TC70	Modification of node 38. The text `FILENAME` is changed to a different label.	38	`EnterpriseManagementException`: JSON does not have the expected structure

Test	Objective / Explanation	Node(s)	Expected result
TC71	Modification of node 39. The closing quotation mark of the `FILENAME` label is replaced with another character.	39	`EnterpriseManagementException`: The file is not JSON formatted
TC72	Modification of node 40. The opening quotation mark of the `FILENAME` value is replaced with another character.	40	`EnterpriseManagementException`: The file is not JSON formatted
TC73	Modification of node 43. The closing quotation mark of the `FILENAME` value is replaced with another character.	43	`EnterpriseManagementException`: The file is not JSON formatted
TC74	Modification of node 44. The file name part is changed to an invalid value.	44	`EnterpriseManagementException`: JSON data has no valid values
TC75	Modification of node 45. The extension `.pdf` is changed to an invalid extension.	45	`EnterpriseManagementException`: JSON data has no valid values
TC76	Duplication of node 46. The extension `.docx` is duplicated.	46	`EnterpriseManagementException`: JSON data has no valid values
TC77	Modification of node 46. The extension `.docx` is changed to an invalid extension.	46	`EnterpriseManagementException`: JSON data has no valid values
TC78	Duplication of node 47. The extension `.xlsx` is duplicated.	47	`EnterpriseManagementException`: JSON data has no valid values
TC79	Modification of node 47. The extension `.xlsx` is changed to an invalid extension.	47	`EnterpriseManagementException`: JSON data has no valid values

2.5 Justification of the syntax analysis test suite

The syntax analysis test suite was designed to cover the valid structure of the input file and to generate representative malformed or invalid variants from the derivation tree. The valid cases ensure that all non-terminal and terminal alternatives are exercised. The invalid cases, generated through duplication, deletion, and modification, verify that the method correctly distinguishes among malformed JSON files, incorrect JSON structure, and invalid JSON data values.

3 Structural Testing

3.1 Testing strategy

In addition to syntax analysis, structural testing was applied to Method 2 in order to analyze the internal control flow of the implementation and ensure path coverage. For this purpose, a flow control graph was built from the source code, the independent basic paths were identified, and additional cases were considered for loops where required.

3.2 Flow control graph

In order to make the flow control graph easier to understand, we first include a numbered version of the source code for Method 2. This numbered code was used as the reference for assigning

node numbers in the graph. Figure 2 shows the final numbering of the relevant statements and decisions in the implementation of `register_document`.

```
def register_document(self, input_file: str):
    """Registers document in the Enterprise Manager"""
    try:
        with open(input_file, "r", encoding="utf-8", newline="") as file:
            input_data = json.load(file)
    except FileNotFoundError as ex:
        raise EnterpriseManagementException("Input file not found") from ex
    except json.JSONDecodeError as ex:
        raise EnterpriseManagementException("The file is not JSON formatted") from ex

    if not isinstance(input_data, dict):
        raise EnterpriseManagementException("The file is not JSON formatted")

    if set(input_data.keys()) != {"PROJECT_ID", "FILENAME"}:
        raise EnterpriseManagementException("JSON does not have the expected structure")

    project_id = input_data["PROJECT_ID"]
    filename = input_data["FILENAME"]

    if not isinstance(project_id, str) or not isinstance(filename, str):
        raise EnterpriseManagementException("JSON data has no valid values")

    if not re.fullmatch(r"[a-f0-9]{32}$", project_id):
        raise EnterpriseManagementException("JSON data has no valid values")

    if not re.fullmatch(r"[a-zA-Z0-9]{8}\.(pdf|docx|xlsx)$", filename):
        raise EnterpriseManagementException("JSON data has no valid values")

    try:
        document = ProjectDocument(project_id, filename)
        document_signature = document.document_signature
    except Exception as ex:
        raise EnterpriseManagementException(
            "Internal processing error when getting the file_signature"
        ) from ex

    try:
        with open(ALL_DOCUMENTS_PATH, "r", encoding="utf-8", newline="") as file:
            documents_list = json.load(file)
    except FileNotFoundError:
        documents_list = []

    documents_list.append(document.to_json())

    with open(ALL_DOCUMENTS_PATH, "w", encoding="utf-8", newline="") as file:
        json.dump(documents_list, file, indent=2)

    return document_signature
```

Figure 2: Numbered source code used to identify the nodes of the flow control graph

Figure 3 below shows the complete flow control graph for Method 2. In this graph, all edges have been explicitly represented. The graph includes the branches generated by the three `try/except` blocks, the validation conditions, the internal processing error branch, the branch where `all_documents.json` does not exist, and the final successful execution path.

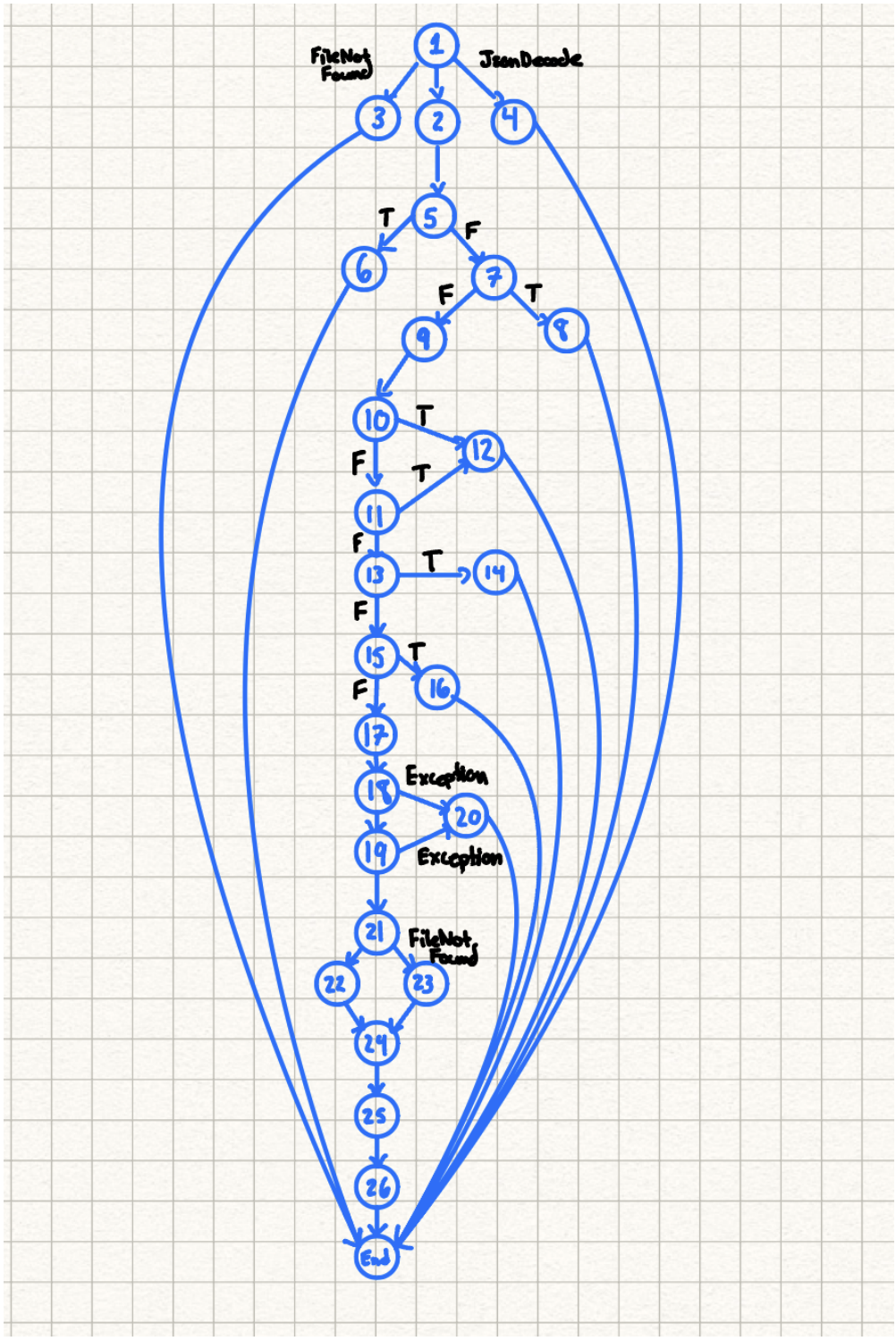

Figure 3: Flow control graph for Method 2

3.3 Basic paths

The Mccabe Complexity was calculated using the formula Mccabe Complexity $= E - N + 2$ where E is the number of edges and N is the number of nodes in the flow control graph. For our graph, we have 37 edges and 27 nodes, including the `end` node. Therefore, the complexity is Mccabe Complexity $= 37 - 27 + 2 = 12$. Thus, according to the structural testing methodology followed in class, 12 independent basic paths must be identified.

- **Path 1:** 1_2_5_7_9_10_11_13_15_17_18_19_21_22_24_25_26_end
- **Path 2:** 1_3_end
- **Path 3:** 1_4_end
- **Path 4:** 1_2_5_6_end
- **Path 5:** 1_2_5_7_8_end
- **Path 6:** 1_2_5_7_9_10_12_end
- **Path 7:** 1_2_5_7_9_10_11_12_end
- **Path 8:** 1_2_5_7_9_10_11_13_14_end
- **Path 9:** 1_2_5_7_9_10_11_13_15_16_end
- **Path 10:** 1_2_5_7_9_10_11_13_15_17_18_20_end
- **Path 11:** 1_2_5_7_9_10_11_13_15_17_18_19_20_end
- **Path 12:** 1_2_5_7_9_10_11_13_15_17_18_19_21_23_24_25_26_end

3.4 Detailed explanation of the 12 structural testing test cases

Table 2 contains the 12 structural testing test cases defined for Method 2. For each test case, we indicate the basic path covered, a short explanation of the objective of the case, and the expected result. For the valid test cases, the SHA-256 signature was omitted from the explanation table, since its full value is too long to display comfortably; however, the correct signatures are included in the Excel file and in the implemented unit tests.

Table 2: Explanation of the 12 structural testing test cases

Test	Basic path	Objective / Explanation	Expected result
TC80	Path 1	Full valid execution path with correct JSON structure and valid values, when `all_documents.json` already exists before running the method. This case covers the complete normal path through the method when the third `try` block reads the file successfully.	Valid SHA-256 signature returned and data stored in `all_documents.json`
TC81	Path 2	Exception path when the input file does not exist. This case forces the first handled exception branch of the initial `try/except` block.	`EnterpriseManagementException: Input file not found`
TC82	Path 3	Exception path when the input file exists but is not valid JSON. This case forces the `JSONDecodeError` branch of the first `try/except` block.	`EnterpriseManagementException: The file is not JSON formatted`

Test	Basic path	Objective / Explanation	Expected result
TC83	Path 4	Exception path when the file is valid JSON but the top-level object is not a dictionary. This case checks the branch corresponding to the first semantic validation after the input file is parsed successfully.	`EnterpriseManagementException: The file is not JSON formatted`
TC84	Path 5	Exception path when the JSON object does not have the expected structure. This case checks the branch where the required keys are not exactly `PROJECT_ID` and `FILENAME`.	`EnterpriseManagementException: JSON does not have the expected structure`
TC85	Path 6	Exception path when `PROJECT_ID` is not of type string. This case covers the first branch of the short-circuit evaluation in the condition that checks whether both fields are strings.	`EnterpriseManagementException: JSON data has no valid values`
TC86	Path 7	Exception path when `FILENAME` is not of type string. This case covers the second branch of the short-circuit evaluation in the same type-checking condition.	`EnterpriseManagementException: JSON data has no valid values`
TC87	Path 8	Exception path when `PROJECT_ID` is a string but does not match the required 32-character hexadecimal format. This case checks the invalid `PROJECT_ID` validation branch.	`EnterpriseManagementException: JSON data has no valid values`
TC88	Path 9	Exception path when `FILENAME` is a string but does not match the required format. This case checks the invalid `FILENAME` validation branch.	`EnterpriseManagementException: JSON data has no valid values`
TC89	Path 10	Exception path when an internal processing error occurs while creating the `ProjectDocument` object. In the implemented test, this path is forced artificially in order to exercise the exception branch of the second `try/except` block.	`EnterpriseManagementException: Internal processing error when getting the file_signature`
TC90	Path 11	Exception path when an internal processing error occurs while obtaining the document signature from the already created `ProjectDocument` object. This case covers the second possible failure inside the second `try/except` block.	`EnterpriseManagementException: Internal processing error when getting the file_signature`

Test	Basic path	Objective / Explanation	Expected result
TC91	Path 12	Full valid execution path with correct JSON structure and valid values, when `all_documents.json` does not exist yet. This case forces the `FileNotFoundError` branch of the third `try/except` block and then continues through the normal write and return path.	Valid SHA-256 signature returned and data stored in `all_documents.json`

3.5 Additional cases for loops

There are no loops in the implementation of Method 2. Therefore, loop testing techniques are not applicable in this case. Since the method does not contain iterative structures such as `for` or `while` loops, there are no loop-specific cases such as zero iterations, one iteration, or multiple iterations to analyze.

3.6 Justification of the structural test suite

The structural testing cases complement the syntax analysis cases by focusing on internal program behavior rather than only on input format. Together, these tests verify that the method not only validates the syntax of the input file, but also executes the expected internal control flow and handles the different exceptional situations correctly.

4 Conclusion

The test design for Method 2 combines syntax analysis and structural testing in order to obtain strong coverage of both the valid input language and the internal control flow of the implementation. The Excel spreadsheet, the implemented Python tests, and this document are aligned so that each test case has a clear purpose and is justified according to the techniques explained in theory class.